COM 141: PROGRAMMING LANGUAGES II (JAVA)

Tourist Destination Catalog System (Kyrgyzstan)

Java (OOP) MySQL (JDBC) File Handling(CSV)

Aizhan Esenbekova

Student Name

COMFCI-24

Group

Problem Statement

Problem

Tourists and users do not have a centralized system to:

- Explore destinations in Kyrgyzstan
- Store travel information
- Manage attractions, hotels, and activities

The problem this project solves is that tourism information in Kyrgyzstan is usually scattered across different sources. There is no single system where users can easily store, update, and manage destinations like attractions, activities, and accommodations.

Existing Issues:

Because of this, users cannot efficiently organize or maintain tourism data. There is also no system that allows both viewing and modifying this information in a structured way.

No structured database

No update/delete system

No unified platform

Project Solution

Developing a Tourist Destination Catalog System that:

✔ Stores destinations in a database
✔ Allows full CRUD operations
✔ Works with CLI + GUI
✔ Supports file storage (backup system)

It also works with a **MySQL database** to ensure that data is permanently stored, even after the program is closed.

In addition, there is both **a command-line interface and a graphical interface** so that the system is more user-friendly and flexible.

The system supports:

- adding new destinations
- viewing all stored data
- updating existing records
- deleting unnecessary records

ID	Type	Name	Location	Description	Extra
2	Attraction	Jeti-Oguz Gorge	Issyk-Kul Regi...	Famous red ro...	Legendary red ...
3	Attraction	Chon-Tuz Salt ...	Kochkor, Naryn	Underground s...	Natural lung the...
4	Attraction	Salkyn-Tor Nat...	Naryn Region	Protected alpin...	Snow leopard ...
5	Attraction	Ala-Archa Natio...	Near Bishkek	Mountain park ...	Easy trekking ...
6	Attraction	Burana Tower	Chuy Valley	Ancient Silk Ro...	Historic tower ...
7	Activity	Horseback Ridi...	Song-Kol Lake	Traditional nom...	Live nomadic c...
8	Activity	Hiking Jeti-Ogu...	Jeti-Oguz Gorge	Mountain hiking...	Photography + ...
9	Activity	Trekking Ala-Ar...	Ala-Archa Natio...	High-altitude tre...	Glaciers + alpin...
10	Activity	Camping at Iss...	Issyk-Kul Lake	Outdoor campi...	Sunsets + swi...

```
=== TOURISM DB MENU ===
1. Add
2. View
3. Update
4. Delete
0. Exit
Choose option: 2
2 | Attraction | Jeti-Oguz Gorge | Issyk-Kul Region | Legendary red cliffs + romantic valley views
3 | Attraction | Chon-Tuz Salt Cave Sanatorium | Kochkor, Naryn | Natural lung therapy inside salt mountains
4 | Attraction | Salkyn-Tor Nature Park | Naryn Region | Snow leopard habitat + untouched nature
5 | Attraction | Ala-Archa National Park | Near Bishkek | Easy trekking + alpine scenery
6 | Attraction | Burana Tower | Chuy Valley | Historic tower + stone balbal statues
7 | Activity | Horseback Riding in Song-Kol | Song-Kol Lake | Live nomadic culture + yurt stays
8 | Activity | Hiking Jeti-Oguz Trails | Jeti-Oguz Gorge | Photography + canyon exploration
9 | Activity | Trekking Ala-Archa Glacier Route | Ala-Archa National Park | Glaciers + alpine peak views
10 | Activity | Camping at Issyk-Kul Beach | Issyk-Kul Lake | Sunsets + swimming + relaxation
18 | Activity | Horseback riding in Tash RabatCaravan Sarai | Tash Rabat | additional Chatyr Kol Lake visit
```

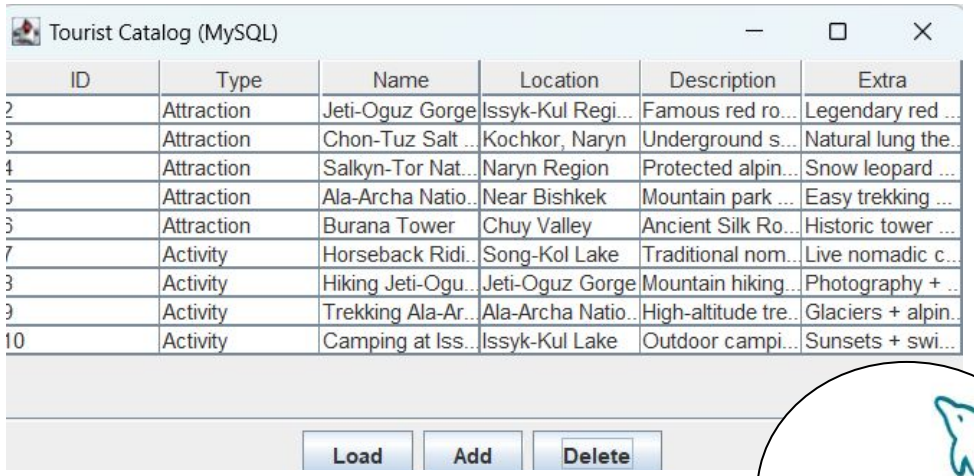
System Design

Presentation Design First is the presentation layer, which includes the CLI and GUI. This is what the user interacts with.

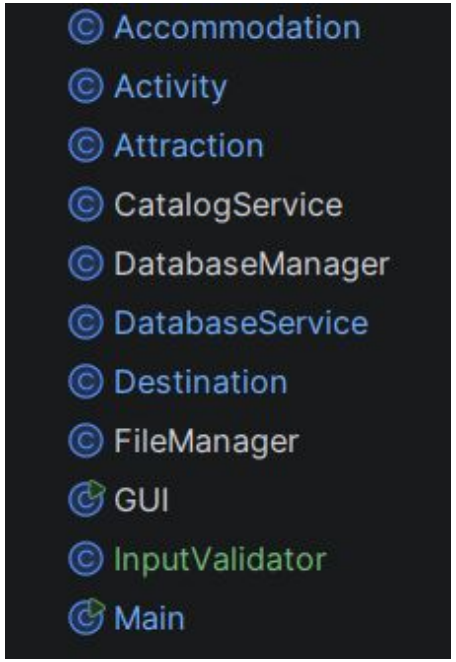
Service Layer: Second is the service layer, which contains the DatabaseService class. This class handles all database operations such as adding, updating, and deleting data.

Data Layer: Third is the data layer, which is the MySQL database itself where all information is stored.

Model Layer: Finally, there is the model layer, which includes classes like Destination, Attraction, Activity, and Accommodation.



ID	Type	Name	Location	Description	Extra
2	Attraction	Jeti-Oguz Gorge	Issyk-Kul Regi...	Famous red ro...	Legendary red ...
3	Attraction	Chon-Tuz Salt ...	Kochkor, Naryn	Underground s...	Natural lung the...
4	Attraction	Salkyn-Tor Nat...	Naryn Region	Protected alpin...	Snow leopard ...
5	Attraction	Ala-Archa Natio...	Near Bishkek	Mountain park ...	Easy trekking ...
6	Attraction	Burana Tower	Chuy Valley	Ancient Silk Ro...	Historic tower ...
7	Activity	Horseback Ridi...	Song-Kol Lake	Traditional nom...	Live nomadic c...
8	Activity	Hiking Jeti-Ogu...	Jeti-Oguz Gorge	Mountain hiking...	Photography + ...
9	Activity	Trekking Ala-Ar...	Ala-Archa Natio...	High-altitude tre...	Glaciers + alpin...
10	Activity	Camping at Iss...	Issyk-Kul Lake	Outdoor campi...	Sunsets + swi...




```
import java.sql.*;
import java.util.Scanner;

public class DatabaseService {
```

CRUD Operations (Create, Read, Update, and Delete)

Create Operation: In this part of the code, I implemented the Create operation, which allows the user to add a new tourist destination into the system. First, I defined an SQL query using a **PreparedStatement:**

```
String sql = "INSERT INTO destinations (type, name, location, description, extra) VALUES (?, ?, ?, ?, ?)";
```

This query inserts a new record into the **destinations table.** The question marks represent placeholders for user input. Then, I used **a PreparedStatement** to safely assign values:

```
stmt.setString(1, type);
stmt.setString(2, name);
stmt.setString(3, location);
stmt.setString(4, desc);
stmt.setString(5, extra);
```

```
=== TOURISM DB MENU ===
1. Add
2. View
3. Update
4. Delete
0. Exit
Choose option: 1
Type (Attraction/Accommodation/Activity): Attraction
Name: Issyk-Kul
Location: Kyrgyzstan, Issyk-kul Region
Description: Beautiful Lake
Extra (type/price/difficulty): Nature
Added successfully!
```

Each value entered by the user is mapped to the corresponding column in the database.

Finally, I executed the query using:

```
stmt.executeUpdate();
```

This sends the data to the MySQL database and creates a new row in the table.
As a result, when the user enters information like Issyk-Kul, it is successfully stored in the database, which can be confirmed both in the console output (“Added successfully!”) and in the MySQL table.

CRUD Operations (Create, Read, Update, and Delete)

```java
public void addDestination(Scanner sc) {  1 usage  aizha
    try (Connection conn = DatabaseManager.connect()) {

        System.out.print("Type (Attraction/Accommodation/Activity): ");
        String type = sc.nextLine();

        System.out.print("Name: ");
        String name = sc.nextLine();

        System.out.print("Location: ");
        String location = sc.nextLine();

        System.out.print("Description: ");
        String desc = sc.nextLine();

        System.out.print("Extra (type/price/difficulty): ");
        String extra = sc.nextLine();

        String sql = "INSERT INTO destinations (type, name, location, description, extra) VALUES (?, ?, ?, ?, ?)";
        PreparedStatement stmt = conn.prepareStatement(sql);

        stmt.setString( parameterIndex: 1, type);
        stmt.setString( parameterIndex: 2, name);
        stmt.setString( parameterIndex: 3, location);
        stmt.setString( parameterIndex: 4, desc);
        stmt.setString( parameterIndex: 5, extra);

        stmt.executeUpdate();

        System.out.println("Added successfully!");

    } catch (Exception e) {
        e.printStackTrace();
```

RESULT

- A new destination is added to the database table.
- The user receives confirmation that the operation was successful.
- The data can be seen later using the view (read) operation or in MySQL.

This line defines the SQL query used to insert a new record into the database table. The question marks are placeholders for user input values.

This creates a PreparedStatement object to execute the SQL query safely and efficiently.

This sets the first parameter in the query, which represents the type of destination (for example, Attraction or Activity).
This assigns the name of the destination entered by the user.
This sets the location where the destination is found.
This adds a description of the destination.
This sets additional information such as price, difficulty level, or category.
This executes the SQL query and inserts the new record into the database.

CRUD Operations (Create, Read, Update, and Delete)

Read Operation: In this part of the code, I implemented the Read operation, which allows the user to view all tourist destinations stored in the system.

First, I defined an SQL query to retrieve all records from the database:

```
String sql = "SELECT * FROM destinations";
```

This query selects all columns and all rows from the **destinations table**. Then, I created a **Statement object** and executed the query:

```
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(sql);
```

The result of this query is stored in a **ResultSet**, which contains all the data returned from the database.
Next, I used a loop to go through each record one by one:

```
while (rs.next()) {
```

Inside the loop, I retrieved the values from each column:

```
rs.getInt("id");
rs.getString("type");
rs.getString("name");
rs.getString("location");
rs.getString("extra");
```

Each of these methods extracts data from the current row in the **ResultSet**.
Finally, I displayed the data to the user using: `System.out.println(...);`

```
=== TOURISM DB MENU ===
1. Add
2. View
3. Update
4. Delete
0. Exit
Choose option: 3
Enter name to update: Issyk-Kul
New name: Issyk-Kul Lake
Updated!
```

CRUD Operations (Create, Read, Update, and Delete)

```java
public void viewAll() {  1 usage  aizha
    try (Connection conn = DatabaseManager.connect()) {

        String sql = "SELECT * FROM destinations";
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql);

        while (rs.next()) {
            System.out.println(
                    rs.getInt( columnLabel: "id") + " | " +
                            rs.getString( columnLabel: "type") + " | " +
                            rs.getString( columnLabel: "name") + " | " +
                            rs.getString( columnLabel: "location") + " | " +
                            rs.getString( columnLabel: "extra")
            );
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

This line defines the SQL query used to retrieve all records from the destinations table. The * symbol means that all columns will be selected.

This creates a Statement object that will be used to execute the SQL query.

This executes the query and stores the result in a ResultSet object. The ResultSet contains all rows returned from the database.

This loop goes through each row in the ResultSet one by one. The method rs.next() moves to the next record and returns true if there is data.

RESULT

- All records stored in the database are displayed to the user.
- The user can see the full list of destinations with their details.
- This confirms that the system successfully reads data from the database.

Exception e represents an error object that is caught when something goes wrong, allowing the program to handle the error instead of crashing + e.printStackTrace() is used to display detailed information about an error

CRUD Operations (Create, Read, Update, and Delete)

Update Operation: In this part of the code, I implemented the Update operation, which allows the user to modify existing tourist destination data in the system.

First, I ask the user to enter the name of the destination they want to update, as well as the new value:

```
System.out.print("Enter name to update: ");
String name = sc.nextLine();
System.out.print("New name: ");
String newName = sc.nextLine();
```

```
=== TOURISM DB MENU ===
1. Add
2. View
3. Update
4. Delete
0. Exit
Choose option: 3
Enter name to update: Issyk-Kul Lake
New name: Issyk-Kul
Updated!
```

This collects the old value (used to find the record) and the new value (used to replace it).Next, I defined an SQL query to update the record:

```
String sql = "UPDATE destinations SET name = ? WHERE name = ?";
```

This query updates the name column in the destinations table. The first ? represents the new value, and the second ? represents the condition (which record to update). Then, I used a **PreparedStatement** to safely assign the values:

```
PreparedStatement stmt = conn.prepareStatement(sql);
stmt.setString(1, newName);
stmt.setString(2, name);
```

```
Choose option: 3
Enter name to update: Eiffel Tower
New name: Tien-Shan
Not found.
```

Here, the new name replaces the old one for the matching record. Finally, I executed the query:

```
int rows = stmt.executeUpdate();
```

This runs the update operation and returns the number of affected rows.

To confirm the result, I check if any row was updated:

```
if (rows > 0)
    System.out.println("Updated!");
else
    System.out.println("Not found.");
```

CRUD Operations (Create, Read, Update, and Delete)

```java
public void update(Scanner sc) {  1 usage  aizha
    try (Connection conn = DatabaseManager.connect()) {

        System.out.print("Enter name to update: ");
        String name = sc.nextLine();

        System.out.print("New name: ");
        String newName = sc.nextLine();

        String sql = "UPDATE destinations SET name = ? WHERE name = ?";
        PreparedStatement stmt = conn.prepareStatement(sql);

        stmt.setString( parameterIndex: 1, newName);
        stmt.setString( parameterIndex: 2, name);

        int rows = stmt.executeUpdate();

        if (rows > 0)
            System.out.println("Updated!");
        else
            System.out.println("Not found.");

    } catch (Exception e) {
        e.printStackTrace();
    }
```

This part asks the user to enter the name of the destination that they want to update. This value is used to find the correct record in the database.

This part takes the new name from the user. It will replace the old value in the database.

This line defines the SQL query used to update a record in the destinations table. The first ? is the new value, and the second ? is the condition used to find the correct record.

This creates a PreparedStatement object that safely sends the SQL query to the database.

This sets the first parameter in the query, which is the new name that will replace the old value.

- RESULT
- All records matching the entered name are updated in the database.
- The user can see the changes immediately when viewing the data again.
- This confirms that the system successfully updates existing records in the database.

CRUD Operations (Create, Read, Update, and Delete)

Delete Operation: In this part of the code, I implemented the Delete operation, which allows the user to remove an existing tourist destination from the system.

```
=== TOURISM DB MENU ===
1. Add
2. View
3. Update
4. Delete
0. Exit
Choose option: 4
Enter name to delete: Issyk-Kul
Deleted!
```

First, I ask the user to enter the name of the destination they want to delete:

```
System.out.print("Enter name to delete: ");
String name = sc.nextLine();
```

This input is used to identify which record should be removed from the database.
Next, I defined the SQL query for deleting a record:

```
String sql = "DELETE FROM destinations WHERE name = ?";
```

This query deletes a row from the **destinations** table where the name matches the user input.
The **?** acts as a **placeholder** for safe input. Then, I used a **PreparedStatement** to assign the value:

```
PreparedStatement stmt = conn.prepareStatement(sql);
stmt.setString(1, name);
```

Here, the entered name is passed into the SQL query, ensuring that the correct record is targeted for deletion. After that, I executed the query:

```
int rows = stmt.executeUpdate();
```

This runs the delete operation and returns the number of rows affected in the database.

```
Choose option: 4
Enter name to delete: Paris
Not found.
```

CRUD Operations (Create, Read, Update, and Delete)

```java
public void delete(Scanner sc) {  1 usage  aizha
    try (Connection conn = DatabaseManager.connect

        System.out.print("Enter name to delete: ");
        String name = sc.nextLine();

        String sql = "DELETE FROM destinations WHERE name = ?";
        PreparedStatement stmt = conn.prepareStatement(sql);
        stmt.setString( parameterIndex: 1, name);

        int rows = stmt.executeUpdate();

        if (rows > 0)
            System.out.println("Deleted!");
        else
            System.out.println("Not found.");

    } catch (Exception e) {
        e.printStackTrace();
```

This part asks the user to enter the name of the destination they want to delete. This value is used to find the correct record in the database.

This line defines the SQL query used to delete a record from the destinations table. The ? is a placeholder for the value that will be provided by the user.

This creates a PreparedStatement object that safely sends the SQL query to the database.

This sets the first parameter in the query, which is the name of the destination that should be deleted.

This executes the delete operation and returns the number of rows affected in the database. And then checks whether the deletion was successful. If at least one row is deleted, it prints “Deleted!”. Otherwise, it shows that the record was not found.

- RESULT
- The selected destination is removed from the database.
- The user can confirm the change by viewing the updated list of destinations.
- This proves that the system successfully deletes existing records from the database.

OOP Principles Requirement

```java
public class Destination {  14 usages  3 inheritors  aizha *
    private String name;  5 usages
    private String location;  4 us
    private String description;
```

```java
public String getName() { return name; }  aizha
public String getLocation() { return location; }  6 usag
public String getDescription() { return description; }
```

Encapsulation: hiding the internal data of a class and controlling access to it through methods (getters and setters).

In my project, encapsulation is used in the Destination class.This means the variable name is private, so it cannot be accessed directly from outside the class. This protects the data from being changed accidentally or incorrectly.
Instead of accessing the variable directly, I use a getter method. This method allows other parts of the program to safely read the value of name without directly accessing the variable itself.

Inheritance: acquires properties and methods from another class (parent class)

```java
public class Attraction extends Destination {
    private String type;  2 usages
```

```java
public Attraction(String name, String location, String description,
    super(name, location, description);
    this.type = type;
```

In my project, I created a parent class called Destination, which contains common attributes shared by all types of tourist destinations. This class contains shared fields such as:

- name
- location
- description

```java
public class Attraction extends Destination {  2 usages  aizha *
    private String type;  2 usages
```

```java
public class Activity extends Destination {
    private String difficulty;  2 usages
```

Polymorphism: same method can behave differently depending on the object that calls it.

In my project, polymorphism is demonstrated using the displayInfo() method, which is defined in the parent class and then overridden in each child class.
First, in the parent class Destination, I define a general method.
Then, in each child class, I override this method to give a different behavior

```java
public void displayInfo() {  2 usages  3 overrides  aizha *
    System.out.println("Destination: " + name);
}
```

```java
@Override  no usages  aizha
public String getType() {
    return "Attraction";
}

@Override  1 usage  aizha
public String toFileFormat() {
    return "Attraction," + getName() + "," + getLocation() + "," + getDescription() + "," + t
}

@Override  2 usages  aizha *
public void displayInfo() {
    System.out.println("🏞 Attraction: " + getName() + " | " + getLocation());
}
```

CLI and GUI Systems

```java
while (true) {
    System.out.println("\n=== TOURISM DB MENU ===");
    System.out.println("1. Add");
    System.out.println("2. View");
    System.out.println("3. Update");
    System.out.println("4. Delete");
    System.out.println("0. Exit");

    int choice = InputValidator.getValidChoice(sc, min: 0, max: 4);

    switch (choice) {
            case 1: service.addDestination(sc); break;
            case 2: service.viewAll(); break;
            case 3: service.update(sc); break;
            case 4: service.delete(sc); break;
            case 0: return;
    }
```

```java
// menu choice validation from 1 to 4
public static int getValidChoice(Scanner sc, int min, int max) {  1 usage  new *
    while (true) {
        try {
            System.out.print("Choose option: ");
            int choice = Integer.parseInt(sc.nextLine());

            if (choice >= min && choice <= max) {
                return choice;
            } else {
                System.out.println("❌ Choose between " + min + " and " + max);
            }

        } catch (Exception e) {
            System.out.println("❌ Invalid input. Enter a number.");
```

```java
private void loadData() {  3 usages  aizha
    try (Connection conn = DatabaseManager.connect()) {

        // ALWAYS clear first
        model.setRowCount(0);

        String sql = "SELECT * FROM destinations";
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql);

        while (rs.next()) {
            model.addRow(new Object[]{
                    rs.getInt( columnLabel: "id"),
                    rs.getString( columnLabel: "type"),
                    rs.getString( columnLabel: "name"),
                    rs.getString( columnLabel: "location"),
                    rs.getString( columnLabel: "description"),
                    rs.getString( columnLabel: "extra")
            });
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
```

Tourist Catalog (MySQL)

ID	Type	Name	Location	Description	Extra
2	Attraction	Jeti-Oguz Gorge	Issyk-Kul Regi...	Famous red ro...	Legendary red ...
3	Attraction	Chon-Tuz Salt ...	Kochkor, Naryn	Underground s...	Natural lung the...
4	Attraction	Salkyn-Tor Nat...	Naryn Region	Protected alpin...	Snow leopard ...
5	Attraction	Ala-Archa Natio...	Near Bishkek	Mountain park ...	Easy trekking ...
6	Attraction	Burana Tower	Chuy Valley	Ancient Silk Ro...	Historic tower ...
7	Activity	Horseback Ridi...	Song-Kol Lake	Traditional nom...	Live nomadic c...
8	Activity	Hiking Jeti-Ogu...	Jeti-Oguz Gorge	Mountain hiking...	Photography + ...
9	Activity	Trekking Ala-Ar...	Ala-Archa Natio...	High-altitude tre...	Glaciers + alpin...
10	Activity	Camping at Iss...	Issyk-Kul Lake	Outdoor campi...	Sunsets + swi...

Load Add Delete

```java
// ACTIONS
loadBtn.addActionListener( ActionEvent e -> loadData());
addBtn.addActionListener( ActionEvent e -> addData());
deleteBtn.addActionListener( ActionEvent e -> deleteData());
```

File Handling (IMPORT / EXPORT)

In my project, I implemented file handling using a CSV file (data.csv) and a separate FileManager class.

To export data, I used:

```java
public class FileManager {  2 usages  aizha
    private static final String FILE_NAME = "data.csv";  2 usages

    public static void save(List<Destination> list) {  1 usage  aizha
        try (PrintWriter writer = new PrintWriter(new FileWriter(FILE_NAME))) {
            for (Destination d : list) {
                writer.println(d.toFileFormat());
            }
            System.out.println("Data saved!");
        } catch (IOException e) {
            System.out.println("Error saving file!");
        }
    }
```

This line creates a PrintWriter object that writes data into a file called data.csv. If the file does not exist, it will be created automatically.

This loop goes through all destinations in the list and writes each one into the file.

- list contains all stored objects in the program
- toFileFormat() converts each object into a CSV format string
- each line is written into the file as one record

To import data:

```java
public static List<Destination> load() {  1 usage  aizha
    List<Destination> list = new ArrayList<>();

    try (BufferedReader reader = new BufferedReader(new FileReader(FILE_NAME))) {
        String line;
        while ((line = reader.readLine()) != null) {
            String[] parts = line.split( regex: ",");
```

This opens the file and prepares it for reading line by line.

This loop reads the file one line at a time until the end of the file. Each line is split into parts using a comma (CSV format).

For example:

Attraction,Issyk-Kul,Kyrgyzstan,Lake,Nature

This creates objects again and adds them back into the program list.

Testing and Error Handling

1. EMPTY INPUT (SPACE ONLY)

Test case:
The user enters only spaces when asked for input (for example name or location).

What happens in code:
In InputValidator, empty input is checked.

```
=== TOURISM DB MENU ===
1. Add
2. View
3. Update
4. Delete
0. Exit
Choose option:
```

```
=== TOURISM DB MENU ===
1. Add
2. View
3. Update
4. Delete
0. Exit
Choose option:
❌ Invalid input. Enter a number.
```

```java
public static String getNonEmptyString(Scanner sc, String message) {
    String input;

    while (true) {
        System.out.print(message);
        input = sc.nextLine().trim();

        if (!input.isEmpty()) {
            return input;
        }
        System.out.println("❌ Input cannot be empty. Try again.");
    }
```

2. RECORD NOT FOUND (UPDATE ERROR)

Test case:
The user tries to update a destination name that does not exist in the database.

```
=== TOURISM DB MENU ===
1. Add
2. View
3. Update
4. Delete
0. Exit
Choose option:
❌ Invalid input. Enter a number.
Choose option: 3
Enter name to update: Issyk-Kul
New name: Issyk-Kul Lake
Not found.
```

3. ADD OPERATION (NORMAL CASE + VALIDATION)

Test case:
The user adds a new destination with correct input.

Valid data is accepted and saved to database

Invalid or empty input is rejected

System successfully adds the record and shows “Added successfully!”

```java
int rows = stmt.executeUpdate();

if (rows > 0)
    System.out.println("Updated!");
else
    System.out.println("Not found.");

catch (Exception e) {
e.printStackTrace();
```

```
=== TOURISM DB MENU ===
1. Add
2. View
3. Update
4. Delete
0. Exit
Choose option: 1
Type (Attraction/Accommodation/Activity): Activity
Name: Horseback riding in Tash RabatCaravan Sarai
Location: Tash Rabat
Description: Exploring ancient stone fortress by horse trekking
Extra (type/price/difficulty): additional Chatyr Kol Lake visit
Added successfully!
```

DATABASE with MySQL

```sql
CREATE TABLE destinations (
    id INT AUTO_INCREMENT PRIMARY KEY,
    type VARCHAR(50),
    name VARCHAR(100),
    location VARCHAR(100),
    description TEXT,
    extra VARCHAR(100)
);
```

The main table is called destinations and it contains:

- id (primary key)
- type
- name
- location
- description
- extra

To connect Java with MySQL, I used JDBC:

```java
    public static Connection connect() throws Exception {  7 usages  aizha
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

This allows Java to communicate with the database.

```java
import java.sql.Connection;
import java.sql.DriverManager;

public class DatabaseManager {  7 usages  aizha
    private static final String URL = "jdbc:mysql://localhost:3306/tourism_db";  1 usage
    private static final String USER = "root";  1 usage
    private static final String PASSWORD = "@izhan5647Gu";  1 usage

    public static Connection connect() throws Exception {  7 usages  aizha
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

Challenges and Conclusion

Problems faced

During the development of this project, I faced several challenges that helped me better understand how real software systems work.

I also faced issues with GUI table refresh in Swing. Sometimes the table did not update after adding or deleting data.

Conclusions

In conclusion, this project demonstrates a complete Tourist Catalog system developed in Java.

It includes CRUD operations for managing destinations, OOP principles for better code structure, MySQL database integration for data storage, CLI and GUI interfaces for user interaction,

In the future, it can be improved by adding authentication, search filters, and a more advanced user interface.